

17ce049(Varun Ladha) Practical 12

In []:

```
from google.colab import drive
drive.mount('/content/drive')
```

In []:

```
import pandas as pd
import numpy as np
import re
import nltk
from nltk.corpus import stopwords
from numpy import array
from keras.preprocessing.text import one_hot
from keras.preprocessing.sequence import pad_sequences
from keras.models import Sequential
from keras.layers.core import Activation, Dropout, Dense
from keras.layers import Flatten
from keras.layers import GlobalMaxPooling1D
from keras.layers.embeddings import Embedding
from sklearn.model_selection import train_test_split
from keras.preprocessing.text import Tokenizer
```

In []:

```
nltk.download('stopwords')
stop_words= set(stopwords.words('english'))
```

[nltk_data] Downloading package stopwords to /root/nltk_data...
 [nltk_data] Unzipping corpora/stopwords.zip.

In []:

```
movie_reviews = pd.read_csv('/content/drive/My Drive/movies-review/IMDB Dataset.csv')
movie_reviews.isnull().values.any()
movie_reviews.shape
movie_reviews.head()
```

Out[5]:

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

In []:



```
movie_reviews.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   review      50000 non-null   object
1   sentiment   50000 non-null   object
dtypes: object(2)
memory usage: 781.4+ KB
```

In []:



```
from nltk.tokenize import sent_tokenize, word_tokenize
from string import punctuation
nltk.download('punkt')
list_of_reviews=[]
sentences = list(movie_reviews['review'])

for sentence in sentences:
    #removal of html tags
    sentence = re.sub('<[^\>]+?>', '', sentence)
    #removal of multiple space
    sentence = re.sub("\s\s+", " ", sentence)
    #Removal of numbers
    sentence = ''.join(c for c in sentence if not c.isdigit())
    #Removal of punctuation
    sentence = ''.join(c for c in sentence if c not in punctuation)

    #tokenized_review
    tokenized_sentence = word_tokenize(sentence.lower())
    #removal of stopwords
    #takes extra time__> new_review = [x for x in tokenized_sentence if x not in stop_words]
    #takes less time if converted to set
    tokenized_sentence=list(set(tokenized_sentence)-stop_words)
    list_of_reviews.append(tokenized_sentence)
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
```

In []:



```
print(list_of_reviews[1])
print(sentences[1])
```

['done', 'wonderful', 'see', 'surface', 'worth', 'technique', 'oldtimebbc', 'halliwell', 'scenes', 'techniques', 'terrificly', 'williams', 'solid', 'comes', 'watching', 'halliwells', 'masters', 'fashion', 'truly', 'piece', 'sense', 'life', 'terribly', 'production', 'use', 'plays', 'gives', 'got', 'murals', 'concerning', 'traditional', 'polari', 'voices', 'comforting', 'every', 'decorating', 'fantasy', 'particularly', 'filming', 'references', 'disappears', 'pat', 'well', 'written', 'chosen', 'diary', 'orton', 'really', 'extremely', 'editing', 'knowledge', 'dream', 'discomforting', 'flat', 'entire', 'realism', 'remains', 'michael', 'guard', 'unassuming', 'seamless', 'home', 'things', 'sheen', 'one', 'comedy', 'little', 'masterful', 'sets', 'rather', 'senses', 'actors', 'entries', 'guided', 'performed', 'great', 'sometimes']

A wonderful little production.

The filming technique is very unassuming- very old-time-BBC fashion and gives a comforting, and sometimes discomforting, sense of realism to the entire piece.

The actors are extremely well chosen- Michael Sheen not only "has got all the polari" but he has all the voices down pat too! You can truly see the seamless editing guided by the references to Williams' diary entries, not only is it well worth the watching but it is a terrificly written and performed piece. A masterful production about one of the great master's of comedy and his life.

The realism really comes home with the little things: the fantasy of the guard which, rather than use the traditional 'dream' techniques remains solid then disappears. It plays on our knowledge and our senses, particularly with the scenes concerning Orton and Halliwell and the sets (particularly of their flat with Halliwell's murals decorating every surface) are terribly well done.

In []:



```
#Stemming using Porter Stemmer
from nltk.stem import PorterStemmer
new_list_of_reviews=[]
porter= PorterStemmer()
for review in list_of_reviews:
    new_review = [porter.stem(word) for word in review]
    new_list_of_reviews.append(new_review)
```

In []:

```
#The new list created after stemming
print(new_list_of_reviews[1])
```

```
['got', 'flat', 'solid', 'fantasi', 'mural', 'michael', 'dream', 'realism',
'piec', 'entir', 'fashion', 'wonder', 'everi', 'truli', 'techniqu', 'home',
'well', 'guard', 'rather', 'oldtimebbc', 'extrem', 'set', 'discomfort', 'dia
ri', 'use', 'chosen', 'surfac', 'film', 'unassum', 'play', 'disappear', 'sea
mless', 'knowledg', 'product', 'great', 'entri', 'give', 'realli', 'perfor
m', 'polari', 'one', 'william', 'sheen', 'worth', 'tradi', 'remain', 'sen
s', 'concern', 'comfort', 'particularli', 'orton', 'sens', 'terrificli', 'wr
itten', 'voic', 'sometim', 'actor', 'guid', 'littl', 'pat', 'decor', 'thin
g', 'terribl', 'master', 'techniqu', 'edit', 'comedi', 'halliwel', 'come',
'master', 'scene', 'see', 'done', 'watch', 'halliwel', 'refer', 'life']
```

In []:

```
y = movie_reviews['sentiment']

y = np.array(list(map(lambda x: 1 if x=="positive" else 0, y)))
y
```

Out[28]:

```
array([1, 1, 1, ..., 0, 0, 0])
```

In []:

```
#
from collections import Counter

counts = Counter(words)
vocab = sorted(counts,key=counts.get, reverse=True)
## Build a dictionary that maps words to integers
vocab_to_int = {word:i for i,word in enumerate(vocab,1)}

## use the dict to tokenize each review in reviews_split
## store the tokenized reviews in reviews_ints
reviews_ints = []
for review in reviews_split:
    reviews_ints.append([vocab_to_int[word] for word in review.split()])
```

In []:

```
reviews = pd.read_csv('/content/drive/My Drive/movies-review/IMDB Dataset.csv')
reviews.head()
```

Out[49]:

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

In []:

```
data = reviews['review']
labels = reviews['sentiment']
```

In []:

```
labels = np.array(list(map(lambda x: 1 if x=="positive" else 0, labels)))
```

In []:

```
from sklearn.utils import shuffle

def prepare_imdb_data(data, labels):
    """Prepare training and test sets from IMDB movie reviews."""
    # Create Partation
    data_train, data_test, labels_train, labels_test = train_test_split(data, labels, test_

    #Shuffle reviews and corresponding Labels within training and test sets
    data_train, labels_train = shuffle(data_train, labels_train)
    data_test, labels_test = shuffle(data_test, labels_test)

    # Return a unified training data, test data, training Labels, test Labels
    return data_train, data_test, labels_train, labels_test
```

In []:

```
train_X, test_X, train_y, test_y = prepare_imdb_data(data, labels)
print("IMDb reviews (combined): train = {}, test = {}".format(len(train_X), len(test_X)))
```

```
IMDb reviews (combined): train = 40000, test = 10000
```

In []:



```
print(train_X[100])
print(train_y[100])
```

This short film that inspired the soon-to-be full length feature - Spatula Madness - is a hilarious piece that contends against similar cartoons yielding multiple writers. The short film stars Edward the Spatula who after being fired from his job, joins in the fight against the evil spoons. This premise allows for some funny content near the beginning, but is barely present for the remainder of the feature. This film's 15-minute running time is absorbed by some odd-ball comedy and a small musical number. Unfortunately not much else lies below it. The plot that is set up doesn't really have time to show. But it's surely follows it plot better than many high-budget Hollywood films. This film is worth watching at least a few times. Take it for what it is, and don't expect a deep story.

0

In []:



```
import nltk
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
from string import punctuation
import re
from bs4 import BeautifulSoup

def review_to_words(review):
    nltk.download("stopwords", quiet=True)
    stemmer = PorterStemmer()

    text = BeautifulSoup(review, "html.parser").get_text() # Remove HTML tags
    text = re.sub(r"^[a-zA-Z0-9]", " ", text.lower()) # Convert to lower case
    words = text.split() # Split string into words
    words = [w for w in words if w not in stopwords.words("english")] # Remove stopwords
    words = [w for w in words if w not in punctuation]
    words = [PorterStemmer().stem(w) for w in words] # stem

    return words
```

In []:



```
review_to_words(train_X[125])
```

Out[61]:

```
['happi',  
'surpris',  
'much',  
'interest',  
'movi',  
'jack',  
'frost',  
'introduc',  
'film',  
'produc',  
'distribut',  
'pix',  
'entertain',  
'without',  
'except',  
'everyth',  
'compani',  
'deal',  
'pure',  
'crap',  
'first',  
'import',  
'never',  
'ever',  
'watch',  
'movi',  
'sober',  
'would',  
'unlik',  
'mani',  
'entertaingli',  
'bad',  
'movi',  
'one',  
'feel',  
'made',  
'intent',  
'bad',  
'get',  
'fake',  
'snowman',  
'alway',  
'shown',  
'briefli',  
'way',  
'move',  
'best',  
'movi',  
'waaaaaaaaay',  
'better',  
'michael',  
'keaton',  
'piec',  
'crap',
```

```
'becuz',  
'made',  
'good',  
'movi',  
'version',  
'bad']
```

In []:

```
def preprocess_data(data_train, data_test, labels_train, labels_test):  
    words_train = [review_to_words(review) for review in data_train]  
    words_test = [review_to_words(review) for review in data_test]  
  
    return words_train, words_test, labels_train, labels_test
```

In []:

```
train_X, test_X, train_y, test_y = preprocess_data(train_X, test_X, train_y, labels_train, labels_test)
```

In []:

```
import os  
import pickle  
  
cache_dir = os.path.join("/content/drive/My Drive/movies-review/cache", "sentiment_analysis")  
cache_file = "preprocessed_data.pkl"  
  
os.makedirs(cache_dir, exist_ok=True)  
  
cache_data = dict(words_train=train_X, words_test=test_X,  
                  labels_train=train_y, labels_test=test_y)  
with open(os.path.join(cache_dir, cache_file), "wb") as f:  
    pickle.dump(cache_data, f)  
print("Wrote preprocessed data to cache file:", cache_file)
```

Wrote preprocessed data to cache file: preprocessed_data.pkl

In []:



```
import numpy as np

def build_vocab(data, vocab_size= 5000):
    word_count = {}
    for sentence in data:
        for word in sentence:
            if word in word_count:
                word_count[word] += 1
            else:
                word_count[word] = 1

    sorted_words = [key for key, value in sorted(word_count.items(), key=lambda word:word[1],

word_dict = {}

    for idx, word in enumerate(sorted_words[:vocab_size-2]):
        word_dict[word] = idx + 2

    return word_dict
```

In []:



```
word_dict = build_vocab(train_X)
```

In []:



```
with open(os.path.join(cache_dir, 'word_dict.pkl'), "wb") as f:
    pickle.dump(word_dict, f)
```

In []:



```
!pwd
```

```
/content/drive/My Drive/movies-review/glove
```

In []:



```
!wget http://nlp.stanford.edu/data/glove.42B.300d.zip glove.zip
```

```
--2020-09-11 17:59:40-- http://nlp.stanford.edu/data/glove.42B.300d.zip (ht
tp://nlp.stanford.edu/data/glove.42B.300d.zip)
Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:80... conne
cted.
HTTP request sent, awaiting response... 302 Found
Location: https://nlp.stanford.edu/data/glove.42B.300d.zip (https://nlp.stan
ford.edu/data/glove.42B.300d.zip) [following]
--2020-09-11 17:59:40-- https://nlp.stanford.edu/data/glove.42B.300d.zip (h
ttps://nlp.stanford.edu/data/glove.42B.300d.zip)
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... conn
ected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: http://downloads.cs.stanford.edu/nlp/data/glove.42B.300d.zip (htt
p://downloads.cs.stanford.edu/nlp/data/glove.42B.300d.zip) [following]
--2020-09-11 17:59:40-- http://downloads.cs.stanford.edu/nlp/data/glove.42
B.300d.zip (http://downloads.cs.stanford.edu/nlp/data/glove.42B.300d.zip)
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.6
4.22
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.6
4.22|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1877800501 (1.7G) [application/zip]
Saving to: 'glove.42B.300d.zip'

glove.42B.300d.zip 100%[=====>] 1.75G 1.97MB/s in 14m 4
1s

2020-09-11 18:14:21 (2.03 MB/s) - 'glove.42B.300d.zip' saved [1877800501/187
7800501]

--2020-09-11 18:14:21-- http://glove.zip/ (http://glove.zip/)
Resolving glove.zip (glove.zip)... failed: Name or service not known.
wget: unable to resolve host address 'glove.zip'
FINISHED --2020-09-11 18:14:21--
Total wall clock time: 14m 41s
Downloaded: 1 files, 1.7G in 14m 41s (2.03 MB/s)
```

In []:



```
!unzip glove.42B.300d.zip
```

```
Archive: glove.42B.300d.zip
  inflating: glove.42B.300d.txt
```

In []:

```
def convert_and_pad(word_dict, sentence, pad=300):
    NOWORD = 0 # We will use 0 to represent the 'no word' category
    INFREQ = 1 # and we use 1 to represent the infrequent words, i.e., words not appearing

    working_sentence = [NOWORD] * pad

    for word_index, word in enumerate(sentence[:pad]):
        if word in word_dict:
            working_sentence[word_index] = word_dict[word]
        else:
            working_sentence[word_index] = INFREQ

    return working_sentence, min(len(sentence), pad)

def convert_and_pad_data(word_dict, data, pad=300):
    result = []
    lengths = []

    for sentence in data:
        converted, leng = convert_and_pad(word_dict, sentence, pad)
        result.append(converted)
        lengths.append(leng)

    return np.array(result), np.array(lengths)
```

In []:

```
train_X, train_X_len = convert_and_pad_data(word_dict, train_X)
test_X, test_X_len = convert_and_pad_data(word_dict, test_X)
```

In []:

```
import pandas as pd

pd.concat([pd.DataFrame(train_y), pd.DataFrame(train_X_len), pd.DataFrame(train_X)], axis=1
          .to_csv(os.path.join(cache_dir, 'train.csv'), header=False, index=False)
```

In []:

```
from numpy import array
from numpy import asarray
from numpy import zeros

embeddings_dictionary = dict()
glove_file = open('glove.42B.300d.txt', encoding="utf8")

for line in glove_file:
    records = line.split()
    word = records[0]
    vector_dimensions = asarray(records[1:], dtype='float32')
    embeddings_dictionary[word] = vector_dimensions
glove_file.close()
```

In []:

```

vocab_size = 5000
embedding_matrix = zeros((vocab_size, 300))
for word, index in word_dict.items():
    embedding_vector = embeddings_dictionary.get(word)
    if embedding_vector is not None:
        embedding_matrix[index] = embedding_vector

```

In []:

```

import tensorflow as tf

model = tf.keras.models.Sequential([ tf.keras.layers.Embedding(vocab_size, 300, weights=[em
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(1024, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')])

```

In []:

```
model.summary()
```

Model: "sequential_14"

Layer (type)	Output Shape	Param #
=====		
embedding_19 (Embedding)	(None, 300, 300)	1500000
flatten_8 (Flatten)	(None, 90000)	0
dense_48 (Dense)	(None, 1024)	92161024
dropout_21 (Dropout)	(None, 1024)	0
dense_49 (Dense)	(None, 512)	524800
dropout_22 (Dropout)	(None, 512)	0
dense_50 (Dense)	(None, 64)	32832
dense_51 (Dense)	(None, 1)	65
=====		
Total params: 94,218,721		
Trainable params: 92,718,721		
Non-trainable params: 1,500,000		

In []:

```
model.compile(optimizer=tf.keras.optimizers.RMSprop(learning_rate=0.001), loss='binary_cross
```

In []:



```
history = model.fit(train_X, train_y, batch_size=128, epochs=10, verbose=1, validation_spli
```

Epoch 1/10

```
282/282 [=====] - 16s 57ms/step - loss: 0.6599 - acc: 0.6613 - val_loss: 0.5062 - val_acc: 0.7725
```

Epoch 2/10

```
282/282 [=====] - 16s 56ms/step - loss: 0.4296 - acc: 0.8096 - val_loss: 0.5155 - val_acc: 0.7630
```

Epoch 3/10

```
282/282 [=====] - 16s 56ms/step - loss: 0.2746 - acc: 0.8904 - val_loss: 0.5397 - val_acc: 0.7782
```

Epoch 4/10

```
282/282 [=====] - 16s 56ms/step - loss: 0.1611 - acc: 0.9412 - val_loss: 0.6316 - val_acc: 0.7660
```

Epoch 5/10

```
282/282 [=====] - 16s 56ms/step - loss: 0.0938 - acc: 0.9681 - val_loss: 0.7796 - val_acc: 0.7653
```

Epoch 6/10

```
282/282 [=====] - 16s 56ms/step - loss: 0.0683 - acc: 0.9781 - val_loss: 0.9797 - val_acc: 0.7650
```

Epoch 7/10

```
282/282 [=====] - 16s 56ms/step - loss: 0.0511 - acc: 0.9834 - val_loss: 1.1770 - val_acc: 0.7655
```

Epoch 8/10

```
282/282 [=====] - 16s 56ms/step - loss: 0.0426 - acc: 0.9861 - val_loss: 1.0867 - val_acc: 0.7645
```

Epoch 9/10

```
282/282 [=====] - 16s 56ms/step - loss: 0.0378 - acc: 0.9878 - val_loss: 1.0223 - val_acc: 0.7703
```

Epoch 10/10

```
282/282 [=====] - 16s 56ms/step - loss: 0.0340 - acc: 0.9887 - val_loss: 1.1093 - val_acc: 0.7700
```

In []:



```
score = model.evaluate(test_X, test_y, verbose=1)
print("Test Score:", score[0])
print("Test Accuracy:", score[1])
```

```
313/313 [=====] - 2s 6ms/step - loss: 1.1469 - acc: 0.7539
```

Test Score: 1.1469156742095947

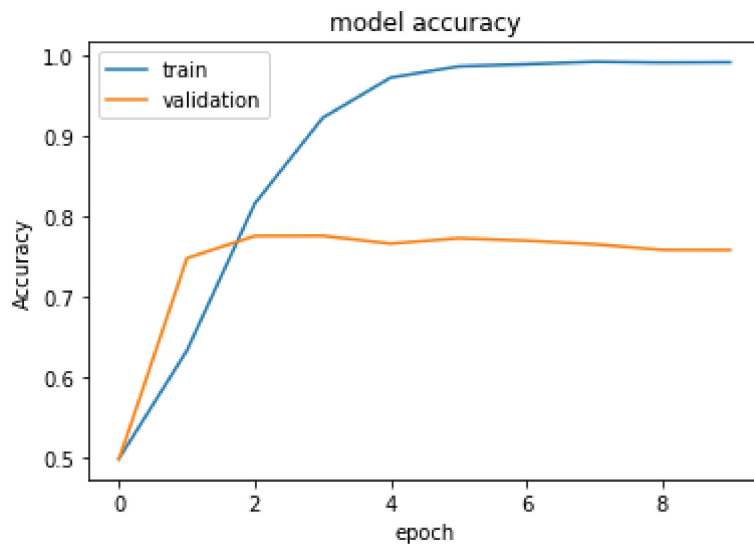
Test Accuracy: 0.7538999915122986

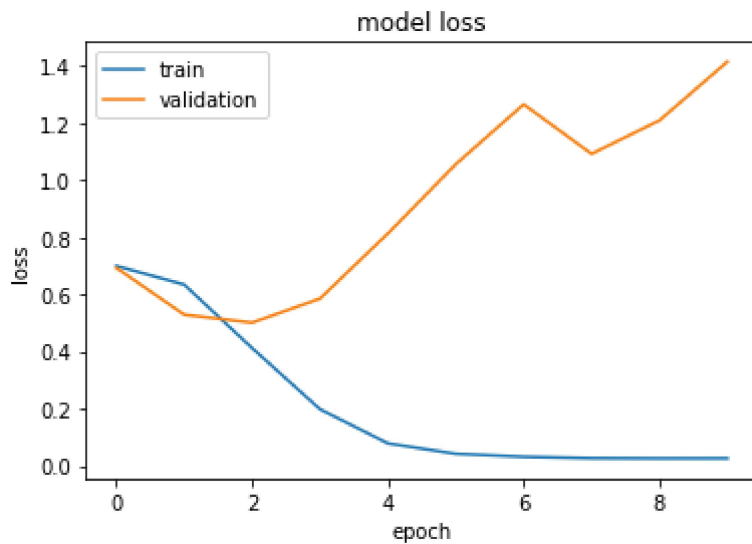
In []:

```
import matplotlib.pyplot as plt

plt.plot(history.history['acc'])
plt.plot(history.history['val_acc'])
plt.title('model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'validation'], loc='upper left')
plt.show()
```





In []:



```
def predict(text):
    words = review_to_words(text)
    text, len = convert_and_pad_data(word_dict, words, 300)
    if (model.predict(text)>0.5).astype("int32")[0][0]:
        return "Positive"
    else:
        return "Negative"
```

In []:



```
text = input("Enter Review:")
print(predict(text))
```

Enter Review:This movie was ok but it was not worth enough for the price we are paying
Positive

In []:



```
-----
NameError                                Traceback (most recent call last)
<ipython-input-146-057e7a1f62de> in <module>()
----> 1 reiew_to_word('This is Good')

NameError: name 'reiew_to_word' is not defined
```

In []:



